

Computational Thinking and the Foundation of Artificial Intelligence

Data, search and problem solving in intelligent engineering
systems

Dr. Elena Fedorchenko, Associate Professor

CUMT

2026–2027



References

- Lectures: https://github.com/ElleDV/CUMT_CTandAI
- Grades:



Sources

- *Michael Negnevitsky. Artificial Intelligence: A Guide to Intelligent Systems (3d Edition), 2011*
- *Course on Foundations of Artificial Intelligence by Dr. Ryan Urbanowicz, University of Pennsylvania (2020)*
(<https://www.youtube.com/watch?v=NVYSnFZwMKk&list=PLafPhSv10SDew4qhV9YeNFwf-1eU0NHng>)
- Other

Course structure

- Introduction
- Computational thinking and intelligent systems
 - Computational thinking
 - Concepts, brief history of AI and building intelligent systems
 - Data, search and problem solving in intelligent engineering systemsZZZ
- Knowledge representation and reasoning
 - Propositional logic for engineering decisions
 - First-order logic and knowledge representation
 - Rule-based expert systems
 - Frame-based expert systems, semantic networks and knowledge graphs
 - Ontologies and structured knowledge
- Search, uncertainty and intelligent decision-making
 - Information retrieval, semantic search and generative AI
 - Uncertainty and probabilistic reasoning
 - Fuzzy logic and fuzzy sets
 - Fuzzy inference and fuzzy expert systems
 - Optimization and evolutionary computation

- ① Introduction
- ② Data for intelligent engineering systems
- ③ Problem solving
- ④ Search
- ⑤ Conclusion

From an engineering problem to an intelligent solution

To solve an engineering problem, an intelligent system needs to:

- represent the situation using data;
- formulate the problem in a computational form;
- search for a suitable solution among possible alternatives.

From an engineering problem to an intelligent solution

Example: Safe route in an underground mine

- Data: tunnel map, worker location, blocked areas, gas levels.
- Problem: Reach a safe exit while respecting safety constraints.
- Search: compare possible routes and find a suitable one.

Data

- What information describes the current situation?

Problem Representation

- How can the engineering task be expressed as states, goals, actions and constraints?

Search

- How can the system efficiently find a path or solution?

- 1 Introduction
- 2 Data for intelligent engineering systems
- 3 Problem solving
- 4 Search
- 5 Conclusion

Why data matter

Intelligent systems use data and/or knowledge to represent the state of the real world and support decisions.

Data sources in mining

- sensors:
 - machine sensors,
 - temperature sensors,
 - vibration sensors;
- cameras;
- equipment and control systems;
- text/documents:
 - geological surveys,
 - technical documents;
- human experts.

Types of data

- numerical,
- categorical,
- time-series,
- image/video,
- text,
- spatial data,
- expert knowledge/rules.

One data source may produce several data types.

Sources and types of mining data

Source	Typical data type	Example
Sensors	Numerical / time-series	CH_4 concentration, temperature, vibration
Cameras	Images / video	Worker Personal Protective Equipment (PPE), equipment condition
Equipment and control systems	Numerical / categorical / time-series data	Operating mode, alarms, maintenance events
Geological surveys	Numerical / spatial data	Rock properties, coordinates, geological measurements
Technical documents	Text data	Safety regulations, maintenance reports, manuals
Human experts	Expert knowledge / rules	Rules for fault diagnosis or safety assessment

The same engineering system may combine several data sources and several data types.

Example

Case: Mine safety assessment.

Source	Data type	Example data
Methane sensor	Time-series	10:00 – 0.4% CH_4 , 10:05 – 0.6% CH_4 , 10:10 – 0.9% CH_4
Camera	Image	Image showing a worker with or without a safety helmet.
Equipment and control system	Categorical	Fan status: ON, Operating mode: Automatic, Alarm: None
Technical documents: maintenance report	Text	"increased vibration was detected during the night shift."

Structured vs unstructured data

Structured data

- Data organized according to a predefined format or scheme, usually in rows, columns or fixed fields:
 - sensor tables,
 - equipment status records,
 - maintenance databases.

Unstructured data

- Data that do not follow a predefined format or scheme, and usually require additional processing to extract useful information:
 - images,
 - video,
 - text reports,
 - audio.

Data quality

Data quality matters:

- missing,
- noisy,
- inconsistent,
- incorrect,
- outdated.

Data quality: example

Case: Fan monitoring

Time	CH_4 concentration, %	Temperature, °C	Fan status
10:00	0.4	22	ON
10:05	—	22	ON
10:10	8.7	23	ON
10:15	0.5	223	ON
10:20	0.5	23	UNKNOWN

Data quality: example

Case: Fan monitoring

Time	CH_4 concentration, %	Temperature, °C	Fan status
10:00	0.4	22	ON
10:05	—	22	ON
10:10	8.7	23	ON
10:15	0.5	223	ON
10:20	0.5	23	UNKNOWN

- Missing data: no CH_4 value at 10:05
- Incorrect data: temperature 223 °C
- Inconsistent data: Fan status "UNKNOWN"

How an intelligent system uses data

Raw data -> data preparation -> analysis -> decision.

Case: Fan monitoring

Raw data:

Time	CH_4 concentration, %	Temperature, °C	Fan status
10:15	0.5	223	ON

Data preparation:

Time	CH_4 concentration, %	Temperature, °C	Fan status
10:15	0.5	23	ON

Analysis: compare CH_4 and temperature with criteria, analyse fan state.

Decision: Condition = Normal.

Data vs Knowledge

- Data: observations or measurements.
- Knowledge: interpreted information, rules, relationships or domain expertise.

Example:

- Data: $CH_4 = 0.5\%$.
- Knowledge: CH_4 above a specified level may indicate a hazardous condition.

- 1 Introduction
- 2 Data for intelligent engineering systems
- 3 Problem solving**
- 4 Search
- 5 Conclusion

From an engineering problem to an intelligent solution

To solve an engineering problem, an intelligent system needs to:

- represent the situation using data;
- formulate the problem in a computational form;
- search for a suitable solution among possible alternatives.

Problem representation

Engineering problems are described in natural language. To solve them computationally, we need a precise representation of the problem.

Example:

Engineering problem: Find a safe route for a worker from the working area to an emergency exit.

Computational representation:

- What is the current situation?
- What is the goal?
- What actions are possible?
- What restrictions must be satisfied?
- How do we evaluate alternative solutions?

Problem formulation

We can specify a computational problem as follows:

Definition

What is the current situation?

What is the goal?

What actions are possible?

What restrictions must be satisfied?

How do we evaluate alternative solutions (what should be optimized)?

How do we evaluate alternative solutions (what measure is used)?

Element

Initial state

Goal state

Actions

Constraints

Objective

Cost

Problem formulation: example

Example:

Engineering problem: Find a safe route for a worker from the working area to an emergency exit.

Definition

What is the current situation?

What is the goal?

What actions are possible?

What restrictions must be satisfied?

How do we evaluate alternative solutions (what should be optimized)?

How do we evaluate alternative solutions (what measure is used)?

Element

Initial state

Goal state

Actions

Constraints

Objective

Cost

Example

Worker at working area A

Emergency exit E

Move through a connected tunnel

Avoid blocked or hazardous tunnels

Find the shortest / fastest / safest route

Distance / travel time / risk

Problem formulation: goal vs objective

Goal: reach the emergency exit.

Objective: decide which route to prefer.

Goal defines what must be achieved, while objective defines what makes one solution better than another.

Problem solution

The same engineering problem can have different solutions depending on the objective.

Objective

Shortest route

Fastest route

Safest route

Solution

minimize distance

minimize travel time

minimize risk

Problem formulation

We can specify a computational problem as follows:

Example:

Engineering problem: Find a safe route for a worker from the working area to an emergency exit.

Definition

What is the current situation?

What is the goal?

What actions are possible?

What restrictions must be satisfied?

How do we evaluate alternative solutions (what should be optimized)?

How do we evaluate alternative solutions (what measure is used)?

Element

Initial state

Goal state

Actions

Constraints

Objective

Cost

Search problem

Search problem

- A search problem is a problem in which we need to find a sequence of actions that transforms an initial state into a goal state.

Initial state -> action 1 -> state 1 -> action 2 -> state 2 -> ...
-> action N -> Goal state

Search problem: state

State

- Representation of the relevant situation at a particular point during problem solving.

Example: current worker location.

State = Junction B

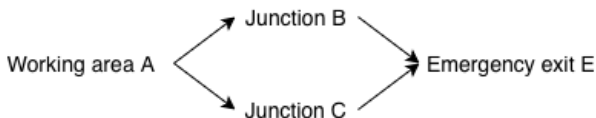
State = {location: B, gas_level: normal, route_open: yes}

Search problem: state space

State space

- The set of all possible states that can be reached while solving a problem.

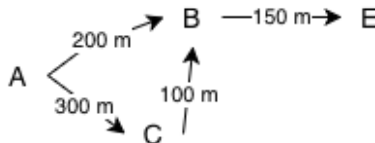
Search explores the state space to find a path from the initial state to a goal state.



Search problem: graph representation

Representing search problem as a graph:

- Nodes represent states.
- Edges represent possible actions or transitions.
- Edge cost represents distance, time, risk, etc.
- Path – a sequence of connected states.



Advantages:

- easy graphical depiction

Problem formulation: task

Example: route to an emergency exit.

Problem: Find a route from working area A to emergence exit E.

Task 1: Formulate this problem using elements below:

- Initial state
- Goal state
- Actions
- Constraints
- Objective
- Cost

Task 2: Represent this problem as a graph.

Problem formulation: example solution

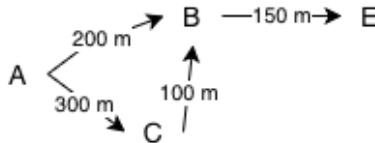
Example: route to an emergency exit.

Problem: Find a route from working area A to emergence exit E.

Task 1: Formulate this problem using elements below:

- Initial state: working area A
- Goal state: emergency exit E
- Actions: move to a connected tunnel section
- Constraints: avoid blocked or hazardous sections
- Objective: minimize distance
- Cost: distance

Task 2: Represent this problem as a graph.

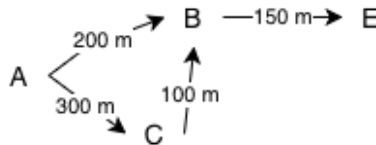


Problem formulation: example solution

Example: route to an emergency exit.

Problem: Find a route from working area A to emergence exit E.

Task 2: Represent this problem as a graph.



Edge	Distance, m	Time, min	Risk
A-B	200	2	Low
B-E	150	1.5	High
A-C	300	4	Low
C-B	150	1.5	Low

What route is better?

From an engineering problem to an intelligent solution

To solve an engineering problem, an intelligent system needs to:

- represent the situation using data;
- formulate the problem in a computational form;
- search for a suitable solution among possible alternatives.

- 1 Introduction
- 2 Data for intelligent engineering systems
- 3 Problem solving
- 4 Search**
- 5 Conclusion

Searching the state space

State space

- The set of all possible states that can be reached while solving a problem.

Once a problem is represented as a state space, the next task is to find a path from the initial state to a goal state.

Search

- The process of exploring possible states and transitions to find a solution.

Exhaustive search

Exhaustive search

- It explores all possible alternatives until a solution is found or all alternatives have been examined.

Advantages:

- can find a solution without additional domain knowledge;
- can find the optimal solution if all relevant alternatives are examined.

Limitation:

- the number of alternatives may become very large.

Exhaustive search may be correct but computationally impractical.

Exhaustive search limitation

Why search become difficult?

If each state has three possible next actions:

- after 1 step we have 3 alternatives;
- after 2 steps we have 9 alternatives;
- after 5 steps we have 243 alternatives;
- after 10 steps we have 59049 alternatives;

It is called exponential growth.

The search space can grow very quickly with problem size.

Heuristics

- Term "heuristic" originates from the Greek verb "heurísko" – to discover or find out.
- The Greek word "heúrēka derives from the same verb – I found it! (*Pinheiro CAR, McNeill F. Heuristics in analytics: a practical perspective of what influences our analytical world. Wiley Online Library. 2014*).
- Heuristics are often characterized as rules of thumb that can be used to speed up the process of decision-making decreasing the search space.
- Heuristics are problem-solving methods that do not guarantee an optimal solution.

Hjeij M., Vilks A. A brief history of heuristics: how did research on heuristics evolve? Humanit Soc Sci Commun 10, 64. 2023

Heuristics: Computer chess

- Chess became a classic testbed for AI problem solving:
 - It is a finite game.
 - A brute-force algorithm going through all possible plays of chess is practically infeasible $\Rightarrow$ need for heuristics.
- 1997: Deep Blue (a chess-playing machine, IBM) defeated G. Kasparov.



Deep Blue – Kasparov Match. 1996

Today's chess-playing machines (AI systems) apply a heuristic approach to eliminate unlikely moves and focus on those with a high probability of defeating their opponent.

*Hjeij M., Vilks A. A brief history of heuristics: how did research on heuristics evolve?
Humanit Soc Sci Commun 10, 64. 2023*

Uninformed vs informed search

Uninformed search

- Uses only the problem definition and doesn't know which state is closer to the goal.

Examples:

- Breadth-first search;
- Depth-first search.

Informed search

- Uses additional information, a heuristic, to guide the search toward more promising states.

Examples:

- Greedy search;
- A* search.

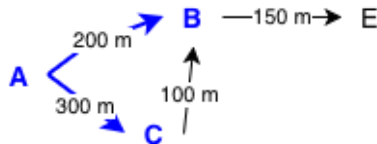
Uninformed search asks: "Where can I go?"

Informed search also asks: "Which option looks more promising?"

Uninformed search

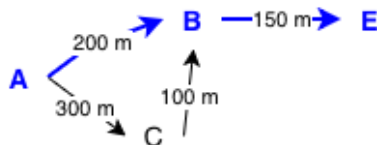
Breadth-first search

- Explores all states at the current level before moving deeper.



Depth-first search

- Follows one path as deeply as possible before returning to try another path.



Informed (heuristic) search

Greedy search

- Chooses the state that appears closest to the goal according to a heuristic.

$h(n)$ – estimated cost from state n to the goal.

Main limitation: a locally attractive choice may lead to a poor overall route.

A* search

- Combines the cost already incurred with an estimate of the remaining cost.

$f(n) = g(n) + h(n)$, where
 $g(n)$ – cost from the start to state n ;

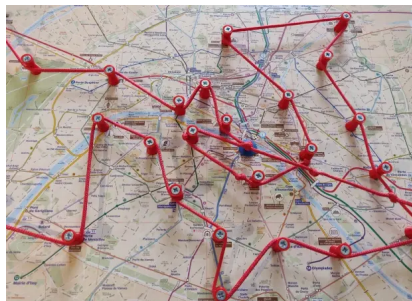
$h(n)$ – estimated cost from n to the goal;

$f(n)$ – estimated total cost of a solution through n .

Heuristics: Shortest path algorithms

1930 – Travelling salesman problem (TSP):

- Several cities with given distances between each two are considered.
- The goal – to find the shortest possible path through all cities and return to the starting point.
- The complexity class NP.



<https://www.routific.com/blog/travelling-salesman-problem>

The greedy (or nearest-neighbour) algorithm (Jon Bentley, 1982): always pick the nearest neighbour as the next city to visit without regard to possible later non-optimal steps.

Hjeij M., Vilks A. A brief history of heuristics: how did research on heuristics evolve? Humanit Soc Sci Commun 10, 64. 2023

Search under engineering constraints

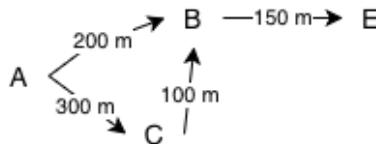
Constraints may include:

- blocked tunnels;
- dangerous gas concentration;
- equipment restrictions;
- maximum travel time;
- inaccessible areas;
- safety regulations.

Objective determines which valid solution is preferred. Constraint determines whether a solution is allowed at all.

Search under engineering constraints

A route may be short, but invalid because it violates a constraint.
Problem: Find a route from working area A to emergence exit E.



Edge	Distance, m	Time, min	Risk
A-B	200	2	Low
B-E	150	1.5	High
A-C	300	4	Low
C-B	150	1.5	Low

Search strategy summary

Different search strategies explore the same problem space in different ways.

Method	Uses heuristic	Main idea
BFS	No	Explore level by level
DFS	No	Explore one branch deeply
Greedy	Yes	Choose what looks closest to the goal
A*	Yes	Balance traveled cost and estimated remaining cost

In engineering applications, the choice of search method depends on the problem size, available information, objectives and constraints.

- ① Introduction
- ② Data for intelligent engineering systems
- ③ Problem solving
- ④ Search
- ⑤ Conclusion

What we covered

- Data for intelligent engineering systems.
- Sources, types and quality of data.
- Problem formulation using states, goals, actions, constraints, and objectives.
- State-space and graph representations.
- Search as a way to find solutions.
- Uninformed and informed search strategies.
- Heuristics and route selection under engineering constraints.

Key takeaways

- Data describe the current situation and provide input for intelligent system.
- A computational problem must be represented clearly before it can be solved.
- Search explores possible states and actions to find a path to a goal.
- The best solution depends on the objective, constraints, and available information.

The next step is to understand how engineering problems are represented using data, search and computational problem-solving methods.

From search to knowledge representation and reasoning

- Data: observations or measurements.
- Knowledge: interpreted information, rules, relationships or domain expertise.

Example:

- Data: $CH_4 = 0.5\%$.
- Knowledge: CH_4 above a specified level may indicate a hazardous condition.

From search to knowledge representation and reasoning

We have represented problems using:

- states,
- actions,
- costs,
- constraints.

Intelligent system also often need to represents:

- facts,
- rules,
- relationships,
- domain knowledge.

Knowledge representation describes what the system knows.

Reasoning describes how the system uses this knowledge to derive conclusions.

Next: Knowledge representation and reasoning.

Thanks!